

MALWARE STATIC ANALYSIS REPORT

(1786629836763_coolapp-desktop-win-34_0_0.exe)

Static Analysis of CoolApp Desktop for Windows

Version 34.0.0

Classification	No Confirmed Malicious Indicators (Static Analysis Only — Payload Not Extracted)
Threat Level	LOW–MEDIUM (static-only; packaged payload behavior unverified)
Analysis Type	Static Analysis Only
Tools	file(1) · GNU objdump · strings · entropy calculation · Authenticode parsing · YARA authoring · Ghidra 12.1
Target SHA-256	46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a

Analyst: Keo Sopharanith | **Instructor:** Kor Sokchea | **Date:** August 16, 2026

Table of Contents

1. Executive Summary
2. File Identification
3. Analysis Scope and Methodology
4. PE Structure Analysis
5. PE Section and Entropy Analysis
6. Overlay and NSIS Payload Analysis
7. Import Table Analysis
8. Critical Imports and Functional Interpretation
9. String Analysis
10. Manifest Analysis
11. Authenticode / Certificate Analysis
12. Byte-Level Evidence
13. Decompilation Analysis and Limitations
14. Attack-Chain Assessment
15. Evasion and Persistence Assessment
16. Indicators of Compromise (IOCs)
17. YARA Detection Rule
18. MITRE ATT&CK Mapping
19. Distribution / Trust Context
20. Risk Assessment
21. Recommendations
22. Conclusion
23. References
24. Next Steps

1. Executive Summary

This report documents a static security/malware analysis of the supplied CoolApp Desktop for Windows installer. The sample is a PE32 Windows GUI executable identified as a Nullsoft Install System (NSIS) self-extracting installer. It contains a large appended installer payload and an embedded Authenticode signature whose signer identity is CoolApp Co., Ltd.

Finding	Result
Threat classification	No confirmed malicious behavior in this static pass
Preliminary risk	LOW–MEDIUM (static-only; payload behavior unverified)
Analysis type	Static analysis; the executable was not run
File format	PE32 / Intel i386 / Windows GUI
Installer framework	Nullsoft Install System v3.04
Code signing	Embedded Authenticode signature; CoolApp Co., Ltd.
Main uncertainty	Compressed/embedded payload was not dynamically unpacked or executed
Recommended next step	Controlled extraction + Windows sandbox + component-level analysis

SHA-256 / File Identification – actual uploaded file

```
$ sha256sum 1786629836763_coolapp-desktop-win-34_0_0.exe
46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a
```

```
$ file 1786629836763_coolapp-desktop-win-34_0_0.exe
PE32 executable for MS Windows 4.00 (GUI), Intel i386, Nullsoft Installer self-extracting archive, 5 sections
```

```
File size: 159,744,936 bytes (152.34 MiB)
MD5: 0f26508283ba34c3f45dc9fcaa6ef837
SHA1: 0e907c5bf35f1bc5d8e498588140b1a456493100
```

Figure 1. Real evidence from the supplied executable: file type, hashes and sample size.

Important: this is not a claim that CoolApp is universally malware-free. It is a statement about the evidence available from this exact installer. Static inspection of an installer stub cannot prove the behavior of every packaged component.

2. File Identification

Property	Value
Filename	1786629836763_coolapp-desktop-win-34_0_0.exe
Product	CoolApp Desktop for Windows
Version	34.0.0
File size	159,744,936 bytes (152.34 MiB)
File type	PE32 executable, Windows GUI
Architecture	Intel i386 / 32-bit
Sections	5
Entry point	0x0040338F
Image base	0x00400000
Installer	Nullsoft Install System v3.04
SHA-256	46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a
MD5	0f26508283ba34c3f45dc9fcaa6ef837
SHA-1	0e907c5bf35f1bc5d8e498588140b1a456493100

The SHA-256 is the preferred identifier for repeatable verification and IOC matching.

3. Analysis Scope and Methodology

The sample was analyzed without execution, using a consistent evidence-first structure: every conclusion in this report is backed by a labeled figure showing the actual bytes, strings, headers, or certificate data observed in the supplied file. Findings are reported only when directly supported by the supplied executable itself or by explicitly identified external sources.

Method / tool	Purpose	Status
file(1)	PE/NSIS file identification	Completed
GNU objdump	PE headers, sections, directories and imports	Completed
strings	Plaintext string search	Completed
Raw-byte inspection	PE/NSIS markers and offsets	Completed
Entropy calculation	Section/payload characterization	Completed
Authenticode structure parsing	Signer, issuer, timestamps	Completed
YARA authoring	Defensive identification rule	Completed
Ghidra	Final application decompilation	Not completed
Dynamic sandbox	Processes, files, registry, network	Not performed
Packet capture	Runtime network endpoints	Not performed

4. PE Structure Analysis

```
PE Header / Structure – actual objdump output

architecture: i386, flags 0x0000010a: EXEC_P, HAS_DEBUG, D_PAGED
start address 0x0040338f
Magic 010b (PE32)
MajorLinkerVersion 6 MinorLinkerVersion 0
AddressOfEntryPoint 0000338f
ImageBase 00400000
Subsystem 00000002 (Windows GUI)
NumberOfSections 5
DllCharacteristics: DYNAMIC_BASE, NX_COMPAT, NO_SEH, TERMINAL_SERVICE_AWARE
Import Directory: present (.rdata)
Security Directory: present (embedded Authenticode signature)
```

Figure 2. Real PE header and security-directory evidence from the supplied file.

PE attribute	Observed	Interpretation
Magic	PE32 (0x010B)	32-bit Windows executable
Machine	Intel i386	x86 32-bit
Entry point	0x0040338F	Normal PE entry point
Image base	0x00400000	Typical PE32 base
SizeOfImage	0x001AB000	Declared image size
Subsystem	Windows GUI (0x2)	Expected for desktop installer
DllCharacteristics	0x8540	Includes DYNAMIC_BASE and NX_COMPAT
Import Directory	RVA 0x8610, size 0xA0	Imports are present
Resource Directory	RVA 0x1A3000, size 0x71C8	Resources/manifest present
Security Directory	File offset 0x09856370, size 0x2038	Embedded Authenticode signature
Relocation Directory	Absent	Noteworthy; runtime ASLR effectiveness needs validation
Debug Directory	Absent	No standard debug directory reported
CLR Header	Absent	Not a .NET managed executable

ASLR/NX flags are positive security characteristics, not evidence of malware. The absent relocation directory is a technical observation that would be worth checking during Windows runtime analysis.

5. PE Section and Entropy Analysis

```
Evidence 7 – Section and overlay entropy analysis

SECTION      RAW SIZE      FILE OFFSET    ENTROPY
.text        0x006627      0x00000400     6.516
.rdata       0x0014A2      0x00006C00     5.249
.data        0x000600      0x00008200     4.037
.ndata       0x128000      0x00000000     N/A
.rsrc        0x0071C8      0x00008800     7.201

NSIS appended payload: 159,672,744 bytes (152.28 MiB)
Payload entropy: 8.000 bits/byte
Interpretation: large/high-entropy appended region is consistent with compressed installer payload.
It is NOT treated as malware evidence by itself.
```

Figure 3. Section layout and entropy calculated from the uploaded executable.

A large, high-entropy appended overlay is not treated as an evasion indicator on its own. Here the evidence the NSIS marker bytes, the ASCII 'NullsoftInst' string, and the entropy profile — is consistent with the expected architecture of a standard NSIS installer, not with a deliberately obfuscated payload.

7. Import Table Analysis

```

Import Table – actual imported Windows APIs

DLL Name: KERNEL32.dll
SetEnvironmentVariableW SetFileAttributesW Sleep GetTickCount
GetFileSize GetModuleFileNameW GetCurrentProcess CopyFileW
CreateThread CreateDirectoryW CreateProcessW RemoveDirectoryW
CreateFileW GetTempFileNameW WriteFile MoveFileExW
GetProcAddress GetModuleHandleA GetExitCodeProcess WaitForSingleObject
DeleteFileW ReadFile LoadLibraryExW SearchPathW

Interpretation: these are broad installer/file/process APIs; by themselves they are not proof of malware.

```

Figure 5. Real representative imported Windows APIs from the supplied CoolApp installer.

DLL	Selected imports	Likely role
KERNEL32.dll	CreateProcessW, CreateFileW, WriteFile, ReadFile, MoveFileExW, DeleteFileW	File installation and process management
KERNEL32.dll	GetTempPathW, GetTempFileNameW	Temporary extraction
KERNEL32.dll	LoadLibraryExW, GetProcAddress	Dynamic library loading
USER32.dll	CreateWindowExW, DialogBoxParamW, MessageBoxIndirectW	Installer GUI
SHELL32.dll	ShellExecuteExW, SHFileOperationW	Shell/file operations
ADVAPI32.dll	RegCreateKeyExW, RegSetValueExW, RegDeleteKeyW	Registry configuration
ADVAPI32.dll	AdjustTokenPrivileges, OpenProcessToken, LookupPrivilegeValueW	Token/privilege operations
ADVAPI32.dll	SetFileSecurityW	File security configuration
ole32.dll	CoCreateInstance	COM operations

8. Critical Imports and Functional Interpretation

Several imported APIs are security-sensitive because malware can abuse them. In an installer, however, the same APIs can be legitimate. Therefore the report does not turn an import into a malicious technique without a confirmed call path.

API	Why analysts care	CoolApp assessment
CreateProcessW	Can spawn payloads or helper processes	Capability only; no child process observed
ShellExecuteExW	Can execute files/commands	Capability only; no malicious command confirmed
AdjustTokenPrivileges	Can alter token privileges	Security-sensitive; no abuse chain

		confirmed
RegCreateKeyExW / RegSetValueExW	Can establish persistence	No persistence artifact confirmed
SetFileSecurityW	Can modify ACLs	Can be legitimate installer behavior
LoadLibraryExW / GetProcAddress	Can dynamically load code	Common Windows capability
ExitWindowsEx	Can restart/log off/shutdown	Worth runtime validation; not malware evidence by itself

9. String Analysis

Authenticode / Manifest / Vendor Strings – actual extracted strings

```
CoolApp Co., Ltd.
Nullsoft Install System v3.04
requestedExecutionLevel level="asInvoker" uiAccess="false"
https://www.ssl.com/repository
SSL.com-SubCA-EV-CodeSigning-RSA-4096-R3
SSL.com-timeStamping-I-RSA-R1
http://ocspssl.com
http://crls.ssl.com/...
```

Interpretation: the executable is an NSIS installer and contains an embedded Authenticode signature chain naming CoolApp Co., Ltd.

Figure 6. Real strings from the supplied binary, including vendor, NSIS and certificate infrastructure.

String category	Observed evidence	Assessment
Vendor	CoolApp Co., Ltd.	Consistent with product identity
Installer	Nullsoft Install System v3.04	Consistent with NSIS
Stub	Nullsoft.NSIS.exehead	NSIS executable stub
Manifest	requestedExecutionLevel level="asInvoker" uiAccess="false"	No explicit admin request
PKI	SSL.com repository / OCSP / CRL references	Consistent with signing chain
Plaintext PowerShell	No obvious malware PowerShell command found in sampled strings	Negative finding only
C2/domain	No confirmed CoolApp C2 domain in plaintext strings	Runtime network remains unknown

The absence of a plaintext downloader/C2 string is useful but not decisive. Compressed or encrypted payloads can hide strings from the stub-level scan.

10. Manifest Analysis

Embedded Application Manifest – actual extracted XML

```
<assemblyIdentity version="1.0.0.0" processorArchitecture="*" name="Nullsoft.NSIS.exehead" type="win32"/>
<description>Nullsoft Install System v3.04</description>
<requestedExecutionLevel level="asInvoker" uiAccess="false"/>
supportedOS entries for modern Windows versions
dpiAware = true
```

Assessment: installer requests normal user-context execution rather than requireAdministrator.

Figure 7. Embedded manifest recovered from the actual CoolApp installer.

Manifest field	Observed	Meaning
assemblyIdentity	Nullsoft.NSIS.exehead	NSIS installer stub
description	Nullsoft Install System v3.04	Installer framework
requestedExecutionLevel	asInvoker	No explicit requireAdministrator request
uiAccess	false	Normal application setting
DPI awareness	true	Normal UI compatibility

This is an important distinction from a malware sample that explicitly requests elevation. The asInvoker setting means Windows does not automatically elevate the installer based on the manifest.

11. Authenticode / Certificate Analysis

Authenticode Certificate Evidence – actual certificate parsing

```

Signer certificate:
C=KH, L=Phnom Penh, O=CoolApp Co., Ltd., OU=IT
CN=CoolApp Co., Ltd., businessCategory=Private Organization, jurisdictionC=KH
Issuer: SSL.com EV Code Signing Intermediate CA RSA R3
Signer serial: 013023BE3E1B8FC236B27390FC2A03F6
Signer validity: May 21 2025 04:00:36 GMT -> May 21 2026 04:00:36 GMT
Signer SHA-256 fingerprint:
89:FF:F2:DB:B2:BB:DB:4D:EA:98:BC:F6:7F:B2:44:C7:F5:7A:30:8F:BE:1C:9C:A7:F3:C0:00:41:4A:27:8D:A0

Embedded timestamp signingTime: 2026-02-10 07:57:41 UTC
Assessment: timestamp predates signer-certificate expiry; signature evidence is consistent with a code-signed
release.

```

Figure 8. Real certificate evidence parsed from the embedded Authenticode signature.

Certificate item	Observed value	Assessment
Subject	C=KH, L=Phnom Penh, O=CoolApp Co., Ltd., OU=IT, CN=CoolApp Co., Ltd.	Matches vendor identity
Issuer	SSL.com EV Code Signing Intermediate CA RSA R3	Code-signing CA chain
Signer validity	2025-05-21 04:00:36 GMT → 2026-05-21 04:00:36 GMT	Certificate validity period
Embedded signing time	2026-02-10 07:57:41 UTC	Before signer expiry
Signer SHA-256 fingerprint	89:FF:F2:DB:B2:BB:DB:4D:EA:98:BC:F6:7F:B2:44:C7:F5:7A:30:8F:BE:1C:9C:A7:F3:C0:00:41:4A:27:8D:A0	Certificate identifier

The signer certificate is expired as of the current analysis date, but the embedded signing time predates the certificate expiry. This report does not claim a full Windows WinVerifyTrust result; final trust validation should be performed on Windows.

12. Byte-Level Evidence

```
PE header:      4D 5A ...          -> MZ
PE signature:   50 45 00 00        -> PE\0\0
NSIS marker:    EF BE AD DE 4E 75 6C 6C 73 6F ... -> EF BE AD DE + 'Nullsoft...'
Security data:  file offset 0x09856370, size 0x2038
```

These byte-level observations connect the high-level conclusions to the actual file contents. The NSIS marker independently supports the installer classification, while the Security Directory establishes that Authenticode data is embedded.

13. Decompilation Analysis and Limitations

Full Ghidra decompilation was not performed in this pass. The supplied file is primarily an NSIS installer stub wrapped around a large compressed payload; the installer stub itself contains little application logic worth decompiling. A complete component-level Ghidra analysis would require safely extracting the packaged application binaries first, then decompiling each extracted component individually.

Analysis item	Status	Reason
Ghidra decompilation of NSIS stub	Not performed	Installer framework is not the final application logic
Extraction of every embedded component	Not performed	Requires dedicated NSIS extraction workflow
Ghidra of final CoolApp binaries	Not performed	Final binaries were not separately supplied
Function call graph of final app	Unavailable	Needs extracted application components
Runtime API tracing	Not performed	Would require sandbox execution
Network capture	Not performed	Would require controlled runtime

This is a limitation, not a missing claim. The report intentionally does not invent Ghidra findings or a function-level attack chain that was not observed.

14. Attack-Chain Assessment

No malicious attack chain was confirmed. The defensible static chain is the installer architecture:

```
User launches CoolApp installer
    ↓
NSIS stub initializes installer
    ↓
Compressed payload is accessed from appended installer data
    ↓
Installer may create files / registry state / helper processes
    ↓
CoolApp components are installed
```

The first three steps are supported by PE/NSIS evidence. The exact files created, subprocesses launched and network connections made remain runtime questions.

15. Evasion and Persistence Assessment

Technique / behavior	Static finding	Assessment
UAC bypass	No bypass mechanism confirmed	LOW
Run/RunOnce persistence	Registry APIs present; no persistence key/value confirmed	UNVERIFIED
Scheduled tasks	No plaintext indicator confirmed	LOW / UNVERIFIED
PowerShell abuse	No confirmed command string	LOW / UNVERIFIED
Anti-debugging	No confirmed mechanism	LOW
Sandbox evasion	No confirmed mechanism	LOW
Dynamic loading	LoadLibraryExW/GetProcAddress present	Generic capability
Privilege adjustment	AdjustTokenPrivileges present	Security-sensitive; no abuse confirmed
Large overlay	High-entropy NSIS payload	Expected installer packaging; not evasion by itself

16. Indicators of Compromise (IOCs)

Evidence 10 – IOC summary from the supplied sample

```
FILE IOC
SHA-256 46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a
MD5 0f26508283ba34c3f45dc9fcaa6ef837
SHA-1 0e907c5bf35f1bc5d8e498588140b1a456493100
Filename 1786629836763_coolapp-desktop-win-34_0_0.exe

NETWORK IOC
No CoolApp C2/domain/IP was confirmed by static string inspection.
SSL.com certificate infrastructure references are signing/PKI infrastructure, not C2.

HOST IOC
No persistence key, dropped payload, scheduled task or child process was observed
because the installer was NOT executed.
```

Figure 9. IOC evidence generated from the exact supplied executable.

16.1 File IOCs

Type	Value
Filename	1786629836763_coolapp-desktop-win-34_0_0.exe
SHA-256	46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a
MD5	0f26508283ba34c3f45dc9fcaa6ef837
SHA-1	0e907c5bf35f1bc5d8e498588140b1a456493100
PE type	PE32 i386 GUI
Installer marker	NullsoftInst

16.2 Network IOCs

Type	Finding
CoolApp C2 domain/IP	None confirmed statically
Outbound endpoint	Not observed; no runtime capture
SSL.com URLs	Certificate/PKI infrastructure only; not classified as C2

16.3 Host IOCs

Type	Finding
Persistence key	Not observed
Dropped executable	Not observed because sample was not executed
Scheduled task/service	Not observed
Child process	Not observed; only API capability exists

17. YARA Detection Rule

This report uses a defensive identification rules built from confirmed byte/string characteristics, rather than a malware-classification rule.

```
rule CoolApp_Windows_Installer_Identification
{
  meta:
    description = "Identifies the analyzed CoolApp Windows NSIS installer"
    author = "Student Analyst"
    date = "2026-08-16"
    sha256 = "46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a"
    severity = "INFO"

  strings:
    $vendor = "CoolApp Co., Ltd." ascii wide
    $nsis   = "Nullsoft Install System v3.04" ascii wide
    $stub   = "Nullsoft.NSIS.exehead" ascii wide
    $ssl    = "SSL.com EV Code Signing Intermediate CA RSA R3" ascii wide

  condition:
    uint16(0) == 0x5A4D and
    2 of ($vendor, $nsis, $stub, $ssl)
}
```

Evidence 9 – Defensive YARA identification rule

```
rule CoolApp_Windows_Installer_Identification
{
  meta:
    description = "Identifies the analyzed CoolApp Windows NSIS installer"
    author = "Student Analyst"
    date = "2026-08-14"
    sha256 = "46bc54e85e6ecc705c4096a7b49696f1604ce6e6bca29b9ec36354360814886a"
    severity = "INFO"

  strings:
    $vendor = "CoolApp Co., Ltd." ascii wide
    $nsis   = "Nullsoft Install System v3.04" ascii wide
    $stub   = "Nullsoft.NSIS.exehead" ascii wide
    $ssl    = "SSL.com EV Code Signing Intermediate CA RSA R3" ascii wide

  condition:
    uint16(0) == 0x5A4D and
    2 of ($vendor, $nsis, $stub, $ssl)
}
```

Figure 10. Defensive YARA rule derived only from confirmed CoolApp strings and the PE MZ header.

Severity is intentionally INFO because the rule identifies the analyzed installer; it does not assert that the installer is malicious.

18. MITRE ATT&CK Mapping

No malicious ATT&CK chain was confirmed for this sample. Generic APIs are not mapped to malicious techniques solely because they could be abused; where a relevant capability exists but no abuse was confirmed, this is marked explicitly as capability-level context rather than a confirmed technique.

Technique	Status	Evidence
T1027 — Obfuscated/Compressed Files	Context only	NSIS compressed installer payload; compression alone is not malicious
T1059 — Command and Scripting Interpreter	Not established	No confirmed command interpreter behavior
T1105 — Ingress Tool Transfer	Not established	No confirmed runtime downloader
T1547 — Boot/Logon Autostart	Not established	No persistence artifact confirmed
T1055 — Process Injection	Not established	No injection behavior observed
T1070 — Indicator Removal	Not established	No cleanup behavior observed
T1548.002 — Bypass UAC	Not established	No bypass chain confirmed

19. Distribution / Trust Context

CoolApp's own support documentation states that its Desktop client supports Windows 10 and 11 and is linked with the mobile client. It also describes Desktop permissions for microphone and camera use for calling. [3][4]

CoolApp's official site states that the client is open source and that its code has been audited by third-party security firms and individual researchers. This is a positive trust signal, but it is not proof that every downloaded binary is safe. [3]

Google Play lists COOLAPP CO., LTD. as the developer and describes the Android client as an independent Signal fork. That supports the vendor identity seen in the Windows certificate, although the Android listing does not independently verify this exact Windows installer. [6]

The privacy policy states that phone numbers are used to provide the service and that this information is end-to-end encrypted. This is application-level policy information and should not be confused with proof of malware status for the Windows executable. [5]

20. Risk Assessment

Risk area	Rating	Reason
Malware behavior	LOW preliminary	No confirmed malicious chain in static evidence
Code-signing identity	POSITIVE	Embedded signer identifies CoolApp Co., Ltd.
Installer complexity	MEDIUM	Large compressed payload requires component analysis
Privilege capability	MEDIUM / UNVERIFIED	Security-sensitive APIs imported
Network behavior	UNVERIFIED	No runtime capture
Supply-chain certainty	MEDIUM / UNVERIFIED	Exact packaged components not separately analyzed
Overall	LOW-MEDIUM preliminary	Positive trust signals, but runtime

		gap remains
--	--	-------------

Final verdict: NO CONFIRMED MALICIOUS INDICATORS IN THIS STATIC PASS. This is not equivalent to a guarantee of safety.

21. Recommendations

- Verify the exact SHA-256 against the official CoolApp distribution/release source.
- Verify the Authenticode signature on Windows with WinVerifyTrust or Get-AuthenticodeSignature.
- Extract the NSIS payload in an isolated environment and inventory every EXE/DLL/script.
- Run the installer in a Windows VM snapshot and record process creation, file writes and registry changes.
- Capture DNS and outbound network connections during installation and first launch.
- Analyze every extracted executable separately with PE tools and Ghidra.
- Check for Run/RunOnce keys, services, scheduled tasks and startup-folder changes after installation.
- Use a reputable multi-engine scanner as an additional independent signal and record the scan date.
- Do not treat code signing alone as proof of safety; combine provenance, static, dynamic and reputation evidence.

22. Conclusion

The supplied CoolApp Desktop Windows installer is structurally consistent with a legitimate NSIS-based desktop application installer. It is a PE32 GUI executable with a large appended installer payload, normal Windows installer-related imports, an asInvoker manifest and an embedded Authenticode signature identifying CoolApp Co., Ltd.

No confirmed malicious payload, plaintext downloader command, persistence mechanism, C2 indicator or anti-analysis chain was identified during this static pass. Security-sensitive APIs are present, but their presence alone does not demonstrate malicious use.

FINAL ASSESSMENT: LOW-MEDIUM PRELIMINARY RISK / NO CONFIRMED MALICIOUS INDICATORS IN STATIC ANALYSIS. The main unresolved question is the behavior of the packaged payload after extraction. A controlled Windows dynamic analysis is required for a stronger final verdict.

23. References

1. Supplied CoolApp Windows installer: 1786629836763_coolapp-desktop-win-34_0_0.exe.
2. CoolApp official website: <https://coolapp.chat/>
3. CoolApp — Installing CoolApp: <https://coolapp.chat/support/installing-coolapp/>
4. CoolApp — Permissions & OS Notification Settings: <https://coolapp.chat/support/coolapp-permissions-os-notification-settings/>
5. CoolApp — Privacy Policy: <https://coolapp.chat/privacy-policy/>
6. Google Play — CoolApp Cambodia developer listing: <https://play.google.com/store/apps/details?id=asia.coolapp.chat>
7. Microsoft PE/COFF specification and Windows Authenticode documentation.

8. Nullsoft Scriptable Install System (NSIS) documentation.
9. MITRE ATT&CK — technique definitions and taxonomy.

24. Next Steps

Priority	Follow-on action	Expected evidence
HIGH	Extract NSIS payload safely	Component inventory
HIGH	Run isolated Windows sandbox	Process/file/registry behavior
HIGH	Capture DNS/TLS traffic	Runtime endpoints
MEDIUM	Ghidra analysis of extracted binaries	Function graph and code paths
MEDIUM	Windows Authenticode verification	Independent trust result
MEDIUM	Component-level hashes	Expanded IOC set
LOW	Create component-specific YARA rules	Better detection coverage

END OF REPORT — STATIC ANALYSIS ONLY